HR EMPLOYEE ATTRITION ANALYSIS

End-to-End People Analytics Documentation

Fresher Edition — Technical Interview Q&A

MySQL | Python | Jupyter | Power BI

1,470	35	25	3	40+
Employees	Features	SQL Queries	Tools	FAQs

IBM HR Analytics Dataset | HR Analytics Project

Document Type: Project Documentation | Version 2.0 | Status: Final | Fresher Technical Edition

TABLE OF CONTENTS

01 PROJECT OVERVIEW

This project delivers a comprehensive end-to-end HR Attrition Analysis built on the IBM HR Analytics dataset. It identifies *which employees are most likely to leave and why*, combining relational database querying, exploratory data analysis, and executive dashboarding.

Layer	Tool	Purpose
Data Wrangling	MySQL 8.0	Explore, clean & aggregate raw HR data
Exploratory Analysis	Python / Jupyter	Visualise distributions, correlations & patterns
Executive Reporting	Power BI	Interactive dashboard for HR decision-makers

Project Objectives

- Identify key drivers of employee attrition using real HR data.
- Quantify attrition rates across departments, job levels, and tenure bands.
- Develop a composite risk score to flag high-risk employees.
- Build an interactive Power BI dashboard for HR leadership.
- Provide fresher-friendly technical Q&A; across the full project tech stack.

02 DATASET DESCRIPTION

The project uses the **IBM HR Analytics — Employee Attrition & Performance** dataset, containing rich demographic, job-related, and satisfaction attributes for 1,470 employees.

Attribute	Detail
Dataset	IBM HR Analytics — Employee Attrition & Performance
Total Records	1,470 employees
Total Columns	35 features
Target Variable	Attrition (Yes = Left / No = Retained)
Raw File	HR-Employee-Attrition.csv
Cleaned File	HR_Employee_Attrition_clean.csv

Key Feature Categories

Category	Features
Demographics	Age, Gender, MaritalStatus, Education, EducationField
Job Details	Department, JobRole, JobLevel, BusinessTravel, OverTime
Compensation	MonthlyIncome, DailyRate, HourlyRate, PercentSalaryHike
Satisfaction	JobSatisfaction, EnvironmentSatisfaction, WorkLifeBalance
Tenure	YearsAtCompany, TotalWorkingYears, YearsInCurrentRole
Performance	PerformanceRating, JobInvolvement, TrainingTimesLastYear

03 PROJECT WORKFLOW

STEP 1 — Data Cleaning

Remove unknown columns · Handle NULL values · Rename tables · Validate data types

STEP 2 — SQL Analysis (MySQL)

25 structured queries · Aggregations & grouping · Window functions · Composite risk scoring

STEP 3 — Python EDA (Jupyter)

Distribution visualisations · Correlation heatmaps · Attrition breakdown charts · Feature analysis

STEP 4 — Power BI Dashboard

KPI cards & metrics · Department drill-through · Interactive slicers · Executive summary view

04 SQL ANALYSIS — 25 QUERIES

All 25 queries are in **HR_Employees.sql**.

Basic Queries (Q1–Q10)

#	Business Question	SQL Concept
Q1	Display first 10 rows	LIMIT
Q2	Total employee count	COUNT(*)
Q3	List unique departments	DISTINCT
Q4	Attrition count breakdown	GROUP BY
Q5	Employees working overtime	WHERE filter
Q6	Average monthly income	AVG()
Q7	Employees with NULL companies worked	IS NULL
Q8	Employee with max monthly income	Subquery
Q9	Employee count by gender	GROUP BY
Q10	Newly joined (0 years tenure)	WHERE filter

Intermediate Queries (Q11–Q20)

#	Business Question	SQL Concept
Q11	Attrition rate % by department	CASE WHEN + ROUND()
Q12	Top 10 by total working years	ORDER BY DESC + LIMIT
Q13	Tenure category buckets	CASE WHEN
Q14	Avg income by job level & attrition	Multi-column GROUP BY
Q15	Top 5 roles with most leavers	Aggregation + LIMIT
Q16	Employees who left within first year	Conditional filter
Q17	Median monthly income	PERCENTILE / subquery
Q18	New compensation after salary hike	Arithmetic expression
Q19	Overtime x attrition cross-count	Cross-grouping
Q20	Top 10 most trained employees	ORDER BY training count

Advanced Queries (Q21–Q25)

#	Business Question	SQL Concept
Q21	Rank employees by experience	RANK() window function
Q22	Top 25% earners per department	NTILE(4) / PERCENTILE

Q23	Attrition rate by income decile	NTILE(10)
Q24	Top 50 high-risk employees	Composite risk score
Q25	Dept + job level summary view	CREATE VIEW

05 PYTHON NOTEBOOK (JUPYTER)

File: **HR_Employees.ipynb**

Section	Description	Output
Data Loading	Import CSV, inspect shape & dtypes	DataFrame summary
Missing Values	Identify & treat null / missing entries	Clean dataset
Distributions	Age, income & tenure histograms	Matplotlib charts
Correlation	Pearson correlation matrix heatmap	Seaborn heatmap
Attrition Breakdown	By department, role, gender & overtime	Bar / pie charts
Income Analysis	Box plots by job level & attrition status	Box plots
WLB & Overtime	Work-life balance vs attrition	Grouped bar chart

```
pip install pandas numpy matplotlib seaborn jupyter
jupyter notebook HR_Employees.ipynb
```

06 POWER BI DASHBOARD

File: **hr_employees.pbix**

Visual	Description
KPI Cards	Total employees · Attrition count · Attrition rate %
Department View	Attrition breakdown: Sales, R&D, HR
Income Analysis	Monthly income by job role & level
Tenure Analysis	Years at company vs. attrition trend
Drill-through	Employee-level detail on click
Slicers	Filter by department, gender, overtime, job level

Open **hr_employees.pbix** in Power BI Desktop. Update data source path if prompted, then click Refresh.

07 REPOSITORY STRUCTURE

File	Type	Description
HR-Employee-Attrition.csv	CSV	Raw IBM HR dataset
HR_Employee_Attrition_clean.csv	CSV	Cleaned & processed dataset
HR_Employees.sql	SQL	25 analytical MySQL queries
HR_Employees.ipynb	Notebook	Jupyter EDA & visualisation
hr_employees.pbix	Power BI	Interactive executive dashboard
README.md	Markdown	Project documentation & setup guide

08 GETTING STARTED

Step 1 — Python

```
pip install pandas numpy matplotlib seaborn jupyter
jupyter notebook HR_Employees.ipynb
```

Step 2 — MySQL

```
CREATE DATABASE employees;
USE employees;

-- Import HR_Employee_Attrition_clean.csv via Workbench Import Wizard
SOURCE /path/to/HR_Employees.sql;
```

Step 3 — Power BI

Open **hr_employees.pbix** in Power BI Desktop, update data source path, then click **Refresh** to populate all visuals.

09 KEY INSIGHTS & FINDINGS

The following insights were derived from the SQL, Python, and Power BI analysis.

~16%	**Overall Attrition Rate** Approximately 237 of 1,470 employees (16.1%) left the organisation.
Sales	**Highest Attrition Dept** Sales department records the highest attrition rate among all departments.
2x	**Overtime Impact** Employees working overtime are approx. twice as likely to leave.
Lower	**Income Gap** Attrited employees earn noticeably lower average monthly income.
1–3 yrs	**Critical Tenure Window** Majority of attrition occurs within the first one to three years.
Low WLB	**Work-Life Balance Driver** Employees with poor work-life balance (score 1) show highest churn.

10 FRESHER Q&A — MySQL & SQL

These are the most commonly asked MySQL and SQL questions for freshers applying to data analyst roles. Every answer is tied directly to this project.

MySQL Basics

1

Q: What is the difference between WHERE and HAVING?

A: WHERE filters rows BEFORE grouping — it works on individual row values. HAVING filters AFTER grouping — it works on aggregated values like COUNT, SUM, AVG. In this project, Query 11 uses WHERE on individual rows and HAVING would be used if we wanted to keep only departments where the attrition rate exceeds a threshold.

```
-- WHERE filters before aggregation
SELECT Department, COUNT(*) FROM hr
WHERE Gender = 'Female' GROUP BY Department;

-- HAVING filters after aggregation
SELECT Department, COUNT(*) FROM hr
GROUP BY Department HAVING COUNT(*) > 100;
```

■ *Pro Tip: Remember: WHERE cannot use aggregate functions. If you see COUNT/AVG in a filter, use HAVING.*

2

Q: What is GROUP BY and when did you use it in this project?

A: GROUP BY collapses multiple rows that share the same value in a column into a single summary row, usually combined with aggregate functions (COUNT, SUM, AVG, MAX, MIN). In this project, it is used in nearly every analytical query — Q4 groups by Attrition to count leavers vs retained; Q9 groups by Gender; Q11 groups by Department to calculate attrition rates.

```
-- Q4: Count employees grouped by attrition status
SELECT Attrition, COUNT(*) AS Total
FROM hr
GROUP BY Attrition;
```

3

Q: What is the difference between COUNT(*) and COUNT(column)?

A: COUNT(*) counts all rows including NULL values. COUNT(column_name) counts only rows where that specific column is NOT NULL. In this project, Q2 uses COUNT(*) to get the total headcount. Q7 finds employees where NumCompaniesWorked IS NULL, which highlights why this distinction matters — NULL rows affect COUNT(column) but not COUNT(*).

```
SELECT COUNT(*) AS Total_Rows,
  COUNT(NumCompaniesWorked) AS Non_Null_Rows
FROM hr;
```

■ *Pro Tip: If a column has NULLs, COUNT(*) and COUNT(col) will give different results. Always check for NULLs first.*

4

Q: What is a subquery? Show an example from this project.

A: A subquery is a SQL query nested inside another query. The inner query runs first and its result is used by the outer query. Query 8 in this project uses a subquery to find the employee with the maximum monthly income — the inner query finds the max, the outer query retrieves the matching employee record.

```
-- Q8: Subquery to find max-income employee
SELECT EmployeeNumber, MonthlyIncome
FROM hr
WHERE MonthlyIncome = (SELECT MAX(MonthlyIncome) FROM hr);
```

■ *Pro Tip: Subqueries can appear in WHERE, FROM (derived table), or SELECT clauses.*

5

Q: What is CASE WHEN and how is it used in attrition analysis?

A: CASE WHEN is SQL's conditional logic — like an IF-ELSE statement. It is the backbone of several queries in this project. Q11 uses it to convert the text value 'Yes'/'No' in the Attrition column into a numeric 1 or 0 so we can SUM it to count leavers. Q24 uses stacked CASE WHEN statements to build a composite risk score.

```
-- Q11: CASE WHEN to count leavers and compute rate
SELECT Department,
  ROUND(SUM(CASE WHEN Attrition='Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*),2)
  AS Attrition_Rate FROM hr GROUP BY Department;
```

MySQL Intermediate

6

Q: What is a window function? How is RANK() used in this project?

A: A window function performs a calculation across a set of rows related to the current row without collapsing them into a single output row (unlike GROUP BY). RANK() assigns a rank number based on ordering. Query 21 ranks all employees by TotalWorkingYears — the most experienced employee gets rank 1 — while keeping all columns intact.

```
-- Q21: Rank employees by experience
SELECT EmployeeNumber, TotalWorkingYears,
  RANK() OVER (ORDER BY TotalWorkingYears DESC) AS ExperienceRank
FROM hr;
```

■ *Pro Tip: Window functions use OVER() clause. They never reduce row count — key difference from GROUP BY.*

7

Q: What is NTILE() and how is it used in salary decile analysis (Q23)?

A: NTILE(n) is a window function that divides rows into n equal-sized buckets and assigns a bucket number to each row. NTILE(10) creates 10 salary deciles (10% bands). Query 23 uses this to segment all employees into income deciles, then calculates the attrition rate for each decile — typically revealing that the lowest income bands have the highest attrition.

```
SELECT income_decile,
  ROUND(SUM(CASE WHEN Attrition='Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*),2)
FROM (
  SELECT Attrition, NTILE(10) OVER (ORDER BY MonthlyIncome) AS income_decile FROM hr
) sub
GROUP BY income_decile;
```

8

Q: What is a VIEW in SQL and why was one created in this project (Q25)?

A: A VIEW is a saved SQL query stored in the database as a virtual table. It does not store data itself — it re-runs the underlying query each time it is accessed. Query 25 creates a summary view showing total employees, leavers, attrition rate, and average income for each Department and JobLevel combination, making it reusable for reporting without rewriting the query each time.

```
-- Q25: Summary VIEW
CREATE VIEW dept_summary AS
SELECT Department, JobLevel, COUNT(*) AS Total,
  SUM(CASE WHEN Attrition='Yes' THEN 1 ELSE 0 END) AS Leavers,
  ROUND(AVG(MonthlyIncome),2) AS Avg_Income
FROM hr GROUP BY Department, JobLevel;
```

■ *Pro Tip: Views are great for standardising reports. Anyone on the team can SELECT * FROM dept_summary.*

9

Q: What is the difference between DELETE and TRUNCATE?

A: DELETE removes specific rows based on a WHERE condition and logs each deletion (can be rolled back). TRUNCATE removes ALL rows from a table instantly without logging individual row deletions (faster, cannot be rolled back in most databases). In this project, we used ALTER TABLE to drop an unwanted column (MyUnknownColumn), not DELETE — an important distinction from removing columns vs removing rows.

```
-- Remove a specific row
DELETE FROM hr WHERE EmployeeNumber = 5;

-- Remove ALL rows instantly
TRUNCATE TABLE hr;
```

■ *Pro Tip: Never TRUNCATE production tables without a backup. DELETE with WHERE is safer for selective removal.*

10

Q: How do you find duplicate records in a table?

A: Group by the columns that should be unique, then use HAVING COUNT(*) > 1 to surface groups with more than one row. In the HR dataset, EmployeeNumber should be unique per employee. This query would catch data quality issues before analysis begins — an important step in data cleaning (Step 1 of this project).

```
-- Find duplicate EmployeeNumbers
SELECT EmployeeNumber, COUNT(*) AS occurrences
FROM hr
GROUP BY EmployeeNumber
HAVING COUNT(*) > 1;
```

11 FRESHER Q&A — PYTHON & PANDAS

Fresher-level Python and Pandas questions based on the EDA performed in HR_Employees.ipynb.

Python & Pandas Basics

11

Q: How do you load the HR CSV file into a pandas DataFrame?

A: Use pandas read_csv() function. After loading, always immediately check the shape, data types, and first few rows to understand your dataset. In this project the cleaned CSV has 1,470 rows and 35 columns after removing the index column.

```
import pandas as pd

df = pd.read_csv('HR_Employee_Attrition_clean.csv')
print(df.shape)        # (1470, 35)
print(df.dtypes)       # column data types
df.head()              # first 5 rows
```

■ *Pro Tip: Always check df.shape and df.dtypes immediately after loading — it catches encoding issues early.*

12

Q: How do you check for and handle missing values in this dataset?

A: Use isnull().sum() to find the count of missing values per column. In this project the Attrition column and most key features have no nulls. However NumCompaniesWorked may have nulls (also checked in SQL Q7). For numerical nulls, use fillna(median) to avoid skewing the data; for categorical nulls, use fillna(mode) or a placeholder.

```
# Check nulls per column
print(df.isnull().sum())

# Fill numerical nulls with median
df['NumCompaniesWorked'].fillna(df['NumCompaniesWorked'].median(), inplace=True)

# Fill categorical nulls with mode
df['Department'].fillna(df['Department'].mode()[0], inplace=True)
```

13

Q: What is the difference between loc and iloc in pandas?

A: loc uses label-based indexing — you provide the actual row/column label. iloc uses integer-based position indexing — you provide the row/column number (0-based). In the HR DataFrame, to select rows where Attrition is 'Yes', you use loc with a boolean condition. To select the first 10 rows by position, you use iloc.

```
# loc: label/condition based
leavers = df.loc[df['Attrition'] == 'Yes']

# iloc: position based (like SQL LIMIT)
first_10 = df.iloc[0:10]

# Select specific column by position
first_col = df.iloc[:, 0]
```

■ *Pro Tip: When in doubt: loc for filtering by values/conditions, iloc for slicing by row/column position numbers.*

14

Q: How do you calculate the attrition rate in Python from this dataset?

A: The Attrition column contains 'Yes'/'No' strings. Convert to 1/0 using map(), then take the mean — which gives the proportion, multiply by 100 for percentage. This mirrors the SQL approach (SUM/COUNT) and confirms the ~16.1% rate found via SQL.

```
# Map Yes/No to 1/0
df['Attrition_Num'] = df['Attrition'].map({'Yes':1, 'No':0})

# Calculate rate
attrition_rate = df['Attrition_Num'].mean() * 100
print(f'Attrition Rate: {attrition_rate:.2f}%')  # ~16.12%
```

15

Q: How do you group and aggregate data in pandas? Compare with SQL GROUP BY.

A: pandas uses groupby() followed by an aggregation function (.count(), .mean(), .sum()). It is the Python equivalent of SQL's GROUP BY + aggregate. The result is a pandas DataFrame (or Series) that mirrors what the SQL query would return.

```
# Equivalent of: SELECT Department, COUNT(*) FROM hr GROUP BY Department
df.groupby('Department').size().reset_index(name='Count')

# Equivalent of Q11 - attrition rate by department
df.groupby('Department')['Attrition_Num'].mean().mul(100).round(2)
```

Python Visualisation

16

Q: What is a correlation heatmap and how do you create one in this project?

A: A correlation heatmap shows pairwise Pearson correlation coefficients between numerical columns as a colour-coded grid. Values close to +1 indicate strong positive correlation, close to -1 means strong negative, and near 0 means no linear relationship. In this project it reveals that MonthlyIncome correlates positively with JobLevel and TotalWorkingYears, which makes intuitive sense.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Only numerical columns
numerical_df = df.select_dtypes(include='number')

plt.figure(figsize=(14, 10))
sns.heatmap(numerical_df.corr(), annot=True, fmt='.2f', cmap='coolwarm')
plt.title('HR Dataset Correlation Heatmap')
plt.tight_layout()
plt.show()
```

■ *Pro Tip: Use annot=True to display correlation values inside each cell of the heatmap.*

17

Q: What is the difference between a bar chart and a histogram?

A: A bar chart is used for categorical data — each bar represents a category (e.g. Department: Sales, R&D;, HR). A histogram is used for continuous numerical data — it groups values into bins (e.g. Age distribution in 5-year bins). In this project, attrition by department uses a bar chart; age or income distribution uses a histogram.

```
# Bar chart - categorical (attrition by department)
df.groupby('Department')['Attrition_Num'].mean().plot(kind='bar', color='#E84393')
plt.title('Attrition Rate by Department')

# Histogram - continuous (age distribution)
df['Age'].plot(kind='hist', bins=20, color='#56C8F5')
plt.title('Employee Age Distribution')
```

18

Q: How do you use value_counts() and what insight did it give in this project?

A: value_counts() counts the frequency of each unique value in a pandas Series. It is the quickest way to understand the distribution of a categorical column. In this project, df['Department'].value_counts() shows that R&D; has the most employees, while df['Attrition'].value_counts() confirms the 237 Yes vs 1233 No split.

```
# Distribution of departments
print(df['Department'].value_counts())
# R&D;          961
# Sales         446
# HR             63

# Attrition split
print(df['Attrition'].value_counts())
# No     1233
# Yes     237
```

■ *Pro Tip: Add normalize=True to get proportions instead of counts: df['Attrition'].value_counts(normalize=True)*

19

Q: How do you filter rows by a condition in pandas? Compare with SQL WHERE.

A: Use boolean indexing with square brackets — pass a condition that evaluates to True/False for each row. This is the pandas equivalent of SQL's WHERE clause. Multiple conditions use & (AND) and | (OR) — always wrap each condition in parentheses.

```
# SQL: SELECT * FROM hr WHERE OverTime = 'Yes'
leavers_ot = df[df['OverTime'] == 'Yes']

# SQL: WHERE Attrition='Yes' AND YearsAtCompany < 2
early_leavers = df[(df['Attrition']=='Yes') & (df['YearsAtCompany']<2)]

print(f'Early leavers: {len(early_leavers)}')
```

■ *Pro Tip: Always use & / | (not 'and'/'or') for multiple conditions in pandas boolean indexing.*

20

Q: How do you export a processed DataFrame to CSV?

A: Use df.to_csv(). Set index=False to avoid writing the pandas row index as an extra column in the output file. This is how the cleaned dataset HR_Employee_Attrition_clean.csv was produced from the raw HR-Employee-Attrition.csv after dropping the MyUnknownColumn (same as what was done in SQL with ALTER TABLE DROP).

```
# Export cleaned DataFrame
df_clean = df.drop(columns=['MyUnknownColumn'], errors='ignore')
df_clean.to_csv('HR_Employee_Attrition_clean.csv', index=False)
print(f'Exported: {df_clean.shape}')
```

12 FRESHER Q&A — POWER BI

Fresher-level Power BI questions based on the hr_employees.pbix dashboard in this project.

Power BI Fundamentals

21

Q: What is the difference between a Measure and a Calculated Column in Power BI?

A: A Calculated Column is computed row-by-row when data is loaded — it adds a new column to the table and is stored in memory. A Measure is computed dynamically at query time based on the current filter context (slicers, page filters). In this dashboard, Attrition Rate % is a Measure because it changes when you filter by department or gender. A static label like EmployeeFullName would be a Calculated Column.

```
-- DAX Measure (recalculates with every filter)
Attrition Rate =
  DIVIDE(COUNTROWS(FILTER(hr, hr[Attrition]="Yes")),
  COUNTROWS(hr), 0) * 100

-- DAX Calculated Column (stored, static)
FullName = hr[FirstName] & " " & hr[LastName]
```

■ *Pro Tip: Rule of thumb: Use Measures for aggregations shown in visuals. Use Calculated Columns for row-level attributes.*

22

Q: What is DAX and why is it important for this HR dashboard?

A: DAX (Data Analysis Expressions) is the formula language used in Power BI to create Measures and Calculated Columns. It is similar to Excel formulas but designed for relational data models. In this HR dashboard, DAX is used to calculate: total employee count (COUNTROWS), attrition rate (DIVIDE + FILTER), average monthly income (AVERAGE), and year-over-year comparisons (CALCULATE with date intelligence functions).

```
-- Key DAX measures used in this dashboard
Total Employees = COUNTROWS(hr)

Attrition Count = CALCULATE(COUNTROWS(hr), hr[Attrition]="Yes")

Avg Monthly Income = AVERAGE(hr[MonthlyIncome])
```

23

Q: What is a Slicer in Power BI and how is it used in this dashboard?

A: A Slicer is an on-canvas filter visual that lets report users interactively filter all other visuals on the page. In this HR dashboard, slicers are provided for Department, Gender, OverTime status, and JobLevel. When a user selects 'Sales' in the Department slicer, every chart on the page automatically updates to show only Sales data — no manual filtering required.

■ *Pro Tip: Slicers in Power BI use the report's filter context — all measures with CALCULATE automatically respect slicer selections.*

24

Q: What is Drill-through in Power BI?

A: Drill-through allows a report user to right-click on a data point (e.g. a department bar) and navigate to a dedicated detail page that shows employee-level information for that selection. In this HR dashboard, drilling through on the 'Sales' bar takes the user to a page showing individual Sales employees, their income, overtime status, and risk score. It is different from Drill-down, which expands a hierarchy within the same visual.

■ *Pro Tip: To set up drill-through: create a detail page, then drag the target field (e.g. Department) into the Drill-through field well.*

25

Q: What is the difference between Import mode and DirectQuery in Power BI?

A: Import mode copies the data into Power BI's in-memory engine (VertiPaq). The report is fast but data is as fresh as the last scheduled refresh. DirectQuery sends every visual interaction as a live query to the source database — always current but slower, especially on large tables. For this HR project, Import mode is used since the dataset is static (1,470 rows) and does not require live updates.

■ *Pro Tip: For reports under 1GB with scheduled refreshes, Import mode is almost always faster and simpler than DirectQuery.*

13 FRESHER Q&A — DATA CONCEPTS & HR ANALYTICS

Core data and HR analytics concepts every fresher should know for interviews.

Data Fundamentals

26

Q: What is attrition rate and how is it calculated?

A: Attrition rate is the percentage of employees who leave an organisation over a given period. Formula: (Number of Leavers / Average Headcount) x 100. In this project: 237 leavers / 1,470 total employees x 100 = 16.1%. This is the single most important KPI tracked on the Power BI dashboard and is the target variable (Attrition) in the entire dataset.

```
-- SQL calculation (Q11 pattern)
SELECT ROUND(SUM(CASE WHEN Attrition='Yes' THEN 1 ELSE 0 END)*100.0/COUNT(*),2)
  AS Overall_Attrition_Rate FROM hr;
-- Result: 16.12%
```

■ *Pro Tip: Industry benchmark for healthy annual attrition is 10–15%. Above 15% signals retention risk.*

27

Q: What is EDA (Exploratory Data Analysis) and what did you do in this project?

A: EDA is the process of analysing a dataset to summarise its main characteristics, often using visual methods, before applying any formal modelling. In this project, EDA covered: checking shape and data types, finding missing values, plotting age/income distributions, creating a correlation heatmap, visualising attrition breakdowns by department and overtime status, and identifying the critical 0–3 year tenure window. EDA answers 'What does this data look like?' before asking 'What can we predict?'

28

Q: What is a NULL value and how was it handled in this project?

A: A NULL value represents missing or unknown data — it is NOT the same as zero or an empty string. NULL does not equal anything, including itself (NULL = NULL is false). In SQL, you must use IS NULL or IS NOT NULL to check for it. This project checks for NULL in NumCompaniesWorked in Query 7. In Python, pandas represents NULL as NaN, checked with isnull() and handled with fillna().

```
-- SQL: Find NULLs
SELECT * FROM hr WHERE NumCompaniesWorked IS NULL;

-- Python: Count NULLs per column
df.isnull().sum()

-- Python: Fill NULLs with median
df['NumCompaniesWorked'].fillna(df['NumCompaniesWorked'].median(), inplace=True)
```

29

Q: What is the difference between a primary key and a foreign key?

A: A Primary Key uniquely identifies each row in a table — it cannot be NULL and must be unique. In this dataset, EmployeeNumber is the primary key. A Foreign Key is a column in one table that references the primary key of another table, creating a relationship between tables. In a multi-table HR system, a Department table's DepartmentID would be a foreign key in the hr_employees table.

```
-- Primary Key example
CREATE TABLE hr (
  EmployeeNumber INT PRIMARY KEY,
  Department VARCHAR(50),
  MonthlyIncome INT
);

-- Foreign Key example
CREATE TABLE dept_details (
  EmployeeNumber INT,
  FOREIGN KEY (EmployeeNumber) REFERENCES hr(EmployeeNumber)
);
```

■ *Pro Tip: Every table should have a primary key. In interviews, always mention that EmployeeNumber is the natural PK in this dataset.*

30

Q: What is the difference between correlation and causation? Give an HR example.

A: Correlation means two variables move together — when one increases, the other tends to increase or decrease. Causation means one variable directly causes the change in another. In this project: OverTime and Attrition are correlated — employees who work overtime have higher attrition. But does overtime CAUSE attrition, or do high-stress roles cause BOTH overtime AND attrition? We cannot determine causation from this observational dataset alone. Making causal claims requires controlled experiments or more advanced causal inference methods.

■ *Pro Tip: In any data interview: always say 'correlation does not imply causation' when discussing variable relationships. It shows analytical maturity.*

HR Attrition Analysis · MySQL · Python · Power BI · IBM HR Analytics · Fresher Technical Edition · 30 Q&As